Session 06

Web Programming

Hands on frontend

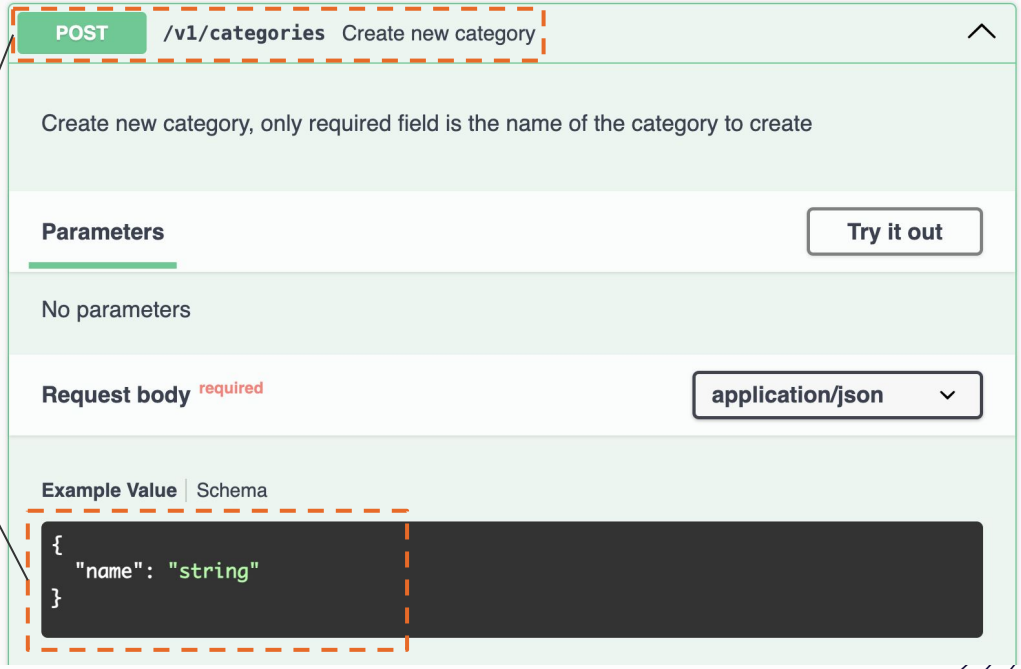
Session 06

Angular and TypeScript Features

Features that will be used for the **second part** of the project.

Extended Types

No need to pass the “id” when creating a new Entity. It will be automatically generated and returned by the backend.



Typescript Types Enhancement

`Omit<Type, Keys>` is a TypeScript utility that **constructs a type** by picking all properties from Type and then removing Keys.

Syntax: `Omit<MyCustomType, "field1" | "field2" | "field3">`

TS *category.ts* ×

src > app > data > TS category.ts > ...

You, last week | 1 author (You)

```
1  export interface Category {
2    id: string
3    name: string
4  }
5
6  export type CategoryCreateInput = Omit<Category, "id">;
```

```
type CategoryCreateInput = {
    name: string;
}
```

Typescript Types Enhancement

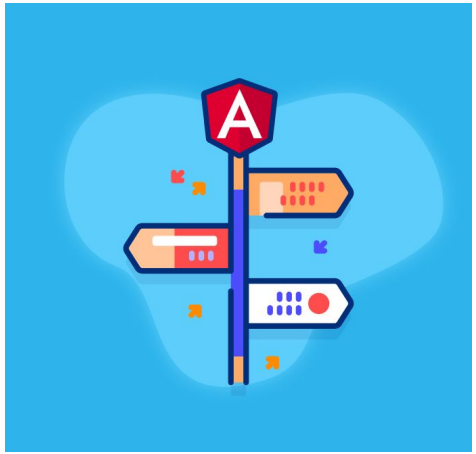
To achieve the opposite of the `Omit` functionality, we can append additional properties to a given type by using the "**&**" symbol followed by the property in a JSON object-like syntax.

```typescript
export interface Category {
  id: string;
  name: string;
}

export type CategoryCreateInputWithIsActive =
Omit<Category, 'id'> & {
  isActive: boolean;
};

const category:
CategoryCreateInputWithIsActive = {
  name: 'Health',
  isActive: true,
};
```

The resulting object will lack the "**id**" property but will gain an additional property named "**isActive**".

Angular Routes & Navigation

- Navigation in Angular enables users to move between different views or pages within an application.
- **Angular Router** provides a robust navigation framework with support for nested routes, lazy loading, and route guards.



Adding a new Route

Adding a new route to an Angular app is straightforward. After making sure that our app imports the `AppRoutingModule` in the **app.module.ts**, we **select or create the component** that will hold the content of our new page. Then, we simply **add the route to the list of routes**, as demonstrated below.

```
const routes: Routes = [
  { path: 'home' , component: AppComponent },
  { path: 'users' , component: UsersListComponent },
  { path: 'users/:id' , component: UserDetailsComponent },
  { path: '', redirectTo: '/home', pathMatch: 'full' },
];
```

Navigation

In order to navigate within our angular app, we should use the `Router` service from `'@angular/router'` which is service that provides navigation among views and URL manipulation capabilities.

Router navigation can be accomplished directly from HTML by utilizing the `[routerLink]` property, as illustrated below:

```
<nav class="navbar bg-body-tertiary">
  <div class="container-fluid">
    <a [routerLink]="['/']" class="navbar-brand">Home</a>
    <a [routerLink]="['/users']" class="navbar-brand">All Users</a>
    <a [routerLink]="['/users', '1']" class="navbar-brand">User 1</a>
    <a [routerLink]="['/users', '2']" class="navbar-brand">User 2</a>
  </div>
</nav>
```

Navigation

Another method to navigate to a specific route is from the TypeScript part. To do so, we inject the Router service into our component and then use the navigation function as shown below:

```
export class CustomComponent {

  @Input()
  user!: User;

  constructor(private router: Router) {}

  goToUserList() {
    this.router.navigate(['users']);
  }

  goToUserDetails() {
    this.router.navigate(['users', this.user.id]);
  }
}
```

Navigation with Parameters

Navigating to a route that takes a parameter will **redirect the page** to a **new URL**. The purpose of passing these parameters is to **retrieve them** in the component and **utilize their values**.

```
export class CustomComponent {

  @Input()
  user!: User;

  constructor(private router: Router) { }

  goToUserDetails(): void {
    this.router.navigate(['users', this.user.id])
  }

}
```

This will open the following route (url):

`/users/3fa85f64-5717-4562-b3fc-2c963f66afa6`

Retrieve Parameters from Current Route

In Angular, **`ActivatedRoute`** is a service that helps you retrieve information about the **current route** in your application. It gives you access to parameters, query parameters, and other information related to the route.

```
export class UserDetailsComponent implements OnInit {

  userId!: string | null;

  constructor(private route: ActivatedRoute) { }

  ngOnInit(): void {
    // Using snapshot
    this.userId = this.route.snapshot.paramMap.get('id');

    // Using paramMap observable
    this.route.paramMap.subscribe(params => {
      this.userId = params.get('id');
    });
  }
}
```

Inject the **`ActivatedRoute`** Service in our destination component.

Two ways to retrieve the id from the url. Either using the **`snapshot`** property or using the **`paramMap`** observable property.

Angular Reactive Forms

Angular forms provide a powerful way to handle user input within applications. A form in Angular consists of various form controls such as input fields, checkboxes, radio buttons, etc., along with validation and submission capabilities. Angular offers two main approaches to building forms: **Template-driven forms** and **Reactive forms**.

Make sure to import the `ReactiveFormsModule` in your `AppModule`

Angular Reactive Forms

The **FormGroup** represents a group of **form controls** and **tracks their values, validity, and state**. Using FormGroup, developers can define complex forms with nested form groups and controls.

Additionally, FormGroup provides methods to handle **form validation and submission**, making it an essential part of building robust forms in Angular applications.
Form groups can be created manually by instantiating instances of **FormGroup** and **FormControl**, or by utilizing the **FormBuilder**, which offers a more streamlined and readable approach.

Angular FormGroup - TypeScript

Using FormGroup and FormControl

```
profileForm = new FormGroup({
  firstName: new FormControl(''),
  lastName: new FormControl(''),
  address: new FormGroup({
    street: new FormControl(''),
    city: new FormControl(''),
    state: new FormControl(''),
    zip: new FormControl(''),
  }),
});
```

Using Form Builder

```
profileForm = this.formBuilder.group({
  firstName: [''],
  lastName: [''],
  address: this.formBuilder.group({
    street: [''],
    city: [''],
    state: [''],
    zip: [''],
  }),
});
```

Angular FormGroup - HTML

```
<form
  [formGroup]="profileForm"
  (ngSubmit)="onSubmit()"
  class="needs-validation"
  autocomplete="off"

>
  <input
    type="text"
    class="form-control"
    formControlName="firstName"
  />
  <!-- Other Form Inputs -->
</form>
```

The instance of the created **FormGroup** with the builder

A custom function that will be executed upon form submission

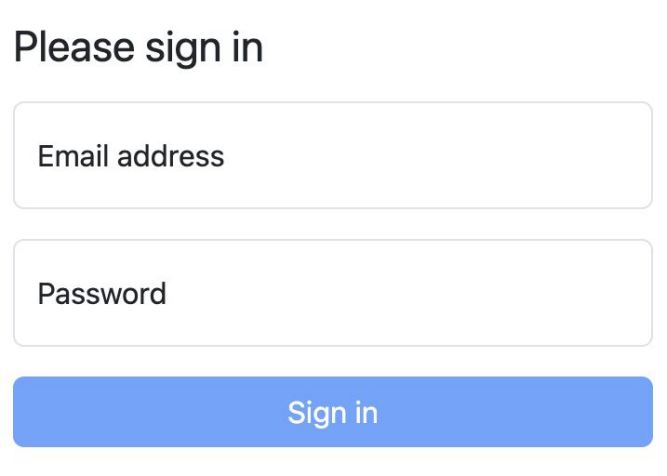
Disable browser input suggestions

Tracks the name of the **FormControl** bound to the directive.

Reactive Forms Validators

You can choose to write your own validator functions, or you can use some of Angular's built-in validators.

Example:




```
form = this.fb.group({
  email: [
    '',
    {
      validators: [Validators.required, Validators.email],
      updateOn: 'blur',
    },
  ],
  password: [
    '',
    [
      Validators.required,
      Validators.minLength(8),
      hasUpperCase(),
      hasLowerCase(),
      hasNumeric(),
    ],
  ],
});

constructor(private fb: FormBuilder) {}
```

Reactive Forms Custom Validators

```
export function hasUpperCase(): ValidatorFn {
  return (control: AbstractControl): ValidationErrors | null => {
    const value = control.value;

    if (!value) {
      return null;
    }

    const hasUpperCase = /[A-Z]+/.test(value);

    return !hasUpperCase ? { missingUpperCase: true } : null;
  };
}
```

```
export function hasLowerCase(): ValidatorFn {
  return (control: AbstractControl): ValidationErrors | null => {
    const value = control.value;

    if (!value) {
      return null;
    }

    const hasLowerCase = /[a-z]+/.test(value);

    return !hasLowerCase ? { missingLowerCase: true } : null;
  };
}
```

The properties that indicates if the validator has error or not.

Make use of the validations in the HTML

To utilize validations and display appropriate error messages, we must create getters in the TypeScript component that return each **FormControl** within our **FormGroup**. Each instance of the **FormControl** contains the **valid**, **invalid** and **errors** properties and much more, which allows us to access any violated validations.

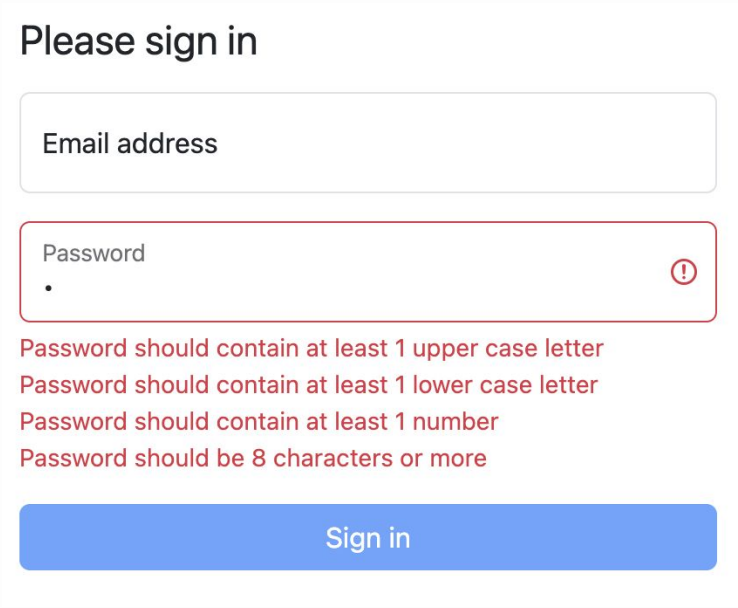
Leveraging Angular's data binding capabilities, we can create dynamic and effective forms.

Example:

```
constructor(private fb: FormBuilder) {}

get email() {
  return this.form.controls['email'];
}

get password() {
  return this.form.controls['password'];
}
```

Make use of the validations in the HTML

```
@if (password.invalid && password.touched) {
  <div class="invalid-feedback text-danger">
    @if (password.errors?.['required']) {
    <div>Password is required</div>
    } @if (password.errors?.['missingUpperCase']) {
    <div>Password should contain at least 1 upper case letter</div>
    } @if (password.errors?.['missingLowerCase']) {
    <div>Password should contain at least 1 lower case letter</div>
    } @if (password.errors?.['missingNumeric']) {
    <div>Password should contain at least 1 number</div>
    } @if (password.errors?.['minlength']) {
    <div>Password should be 8 characters or more</div>
    }
  </div>
```

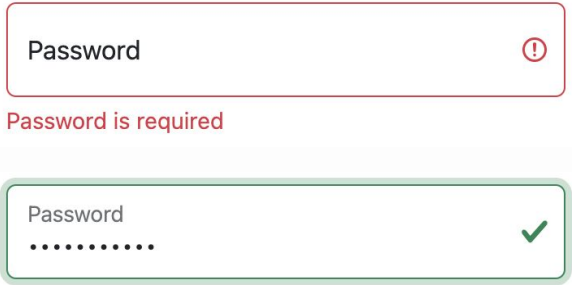


Additional Bootstrap Form Styles

Bootstrap can enhance the user-friendliness of our form elements. By utilizing the **`.is-invalid`** and **`.is-valid`** classes, we can apply styling to form inputs based on their validity.

Here's an example demonstrating this validation in action with an Angular form control:

```
<input
  type="password"
  class="form-control"
  id="floatingPassword"
  placeholder="Password"
  formControlName="password"
  [class.is-invalid]="password?.invalid && password?.touched"
  [class.is-valid]="password?.valid && password?.touched"
/>
```



Reactive Forms Live Demo

You can access a live version of the sign-in example we previously discussed by following this link. Keep it as a resource to assist you in completing your project.

https://stackblitz.com/edit/stackblitz-starters-tm7wxeaj?file=src%2Fmain.ts

💡 **Bonus Tip:** Use **Stackblitz** for quick and efficient demo projects.

Adding an external library to the project

Using **open-source libraries** instead of building from scratch offers **efficiency**, **reliability**, and **community support**. It saves time and resources while tapping into collective expertise. Choosing the right library is vital for **future maintenance**, ensuring **ongoing support** and **scalability**.

GitHub is among several version control platforms where developers share their code, including numerous open-source projects.

However, the most common method for **adding a library** to our project is through a **package manager** like **npm** or **yarn**.

Adding an external library to the project

To add an npm package to a project, follow these steps:

1. **Navigate to Project Directory:**
   Open a terminal or command prompt and navigate to your project directory.

2. **Install Package:**
   Use the npm install command followed by the name of the package you want to install. For example:
   ```
   $ npm install package-name
   ```

3. **Save Package as Development Dependency (Optional):**
   If the package is only needed for development purposes, you can save it as a devDependency using the --save-dev flag:
   ```
   $ npm install package-name --save-dev
   ```

Adding an external library to the project

4. **Verify Installation:**

   Once the installation is complete, verify that the package has been added to your project by checking the **node_modules** directory in your project folder and in your **package.json** dependencies.

5. **Start Using the Package:**

   You can now start using the installed package in your project by importing it into your code.

   Usually, a library comes with a 🚨 **documentation** 🚨 which you can follow to use their code in your project.

BloggerBox Frontend Part 2

BloggerBox Frontend - Part 2

Part 2: Advanced Functionality Integration

- **Post Creation Endpoint Implementation:**
  - Extend the post service functionality to incorporate the creation endpoint for adding new posts.
- **Typescript Types Enhancement:**
  - Integrate Omit and Append features to enhance Typescript types for improved data manipulation.
- **Category Endpoint Integration:**
  - Implement the retrieval of categories through the designated endpoint.
- **New Route Creation:**
  - Establish a new route '/add-post' dedicated to a post creation page, accessible from the navigation bar using the “Write” button.

Sync with Github

BloggerBox Frontend - Part 2

- **Form Design and Validation:**
  - Develop a form with three fields (Title, Category, Content) for post creation, utilizing Bootstrap elements.
  - Implement Reactive Forms Validation to ensure data integrity and user input validation:
    - Title *[text input]*:
      - Required
      - Minimum length of 5 characters
      - Maximum length of 150 characters
    - Category *[**select input**]*:
      - Required
      - Pre-filled with all the tags available on the backend
    - Content *[text input]*:
      - Required
      - Maximum length of 2500 characters
  - Enhance validation UI using Bootstrap for improved user experience.

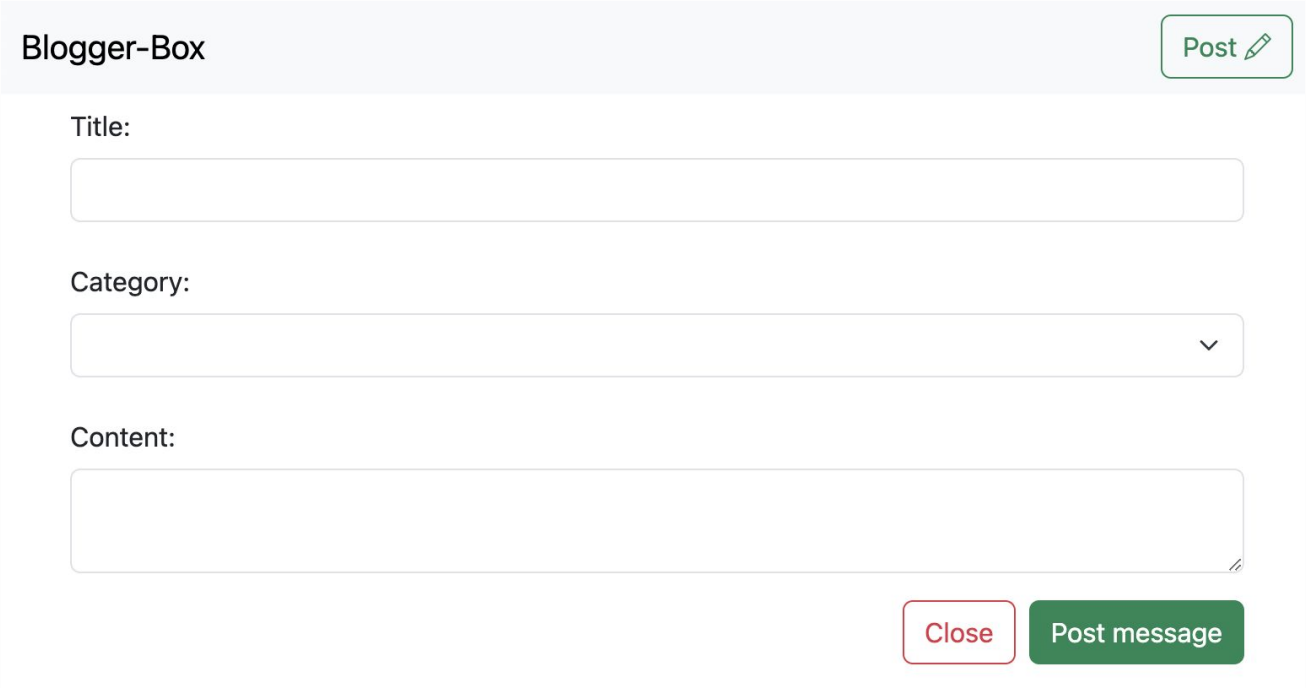
Sync with Github

BloggerBox Frontend - Part 2

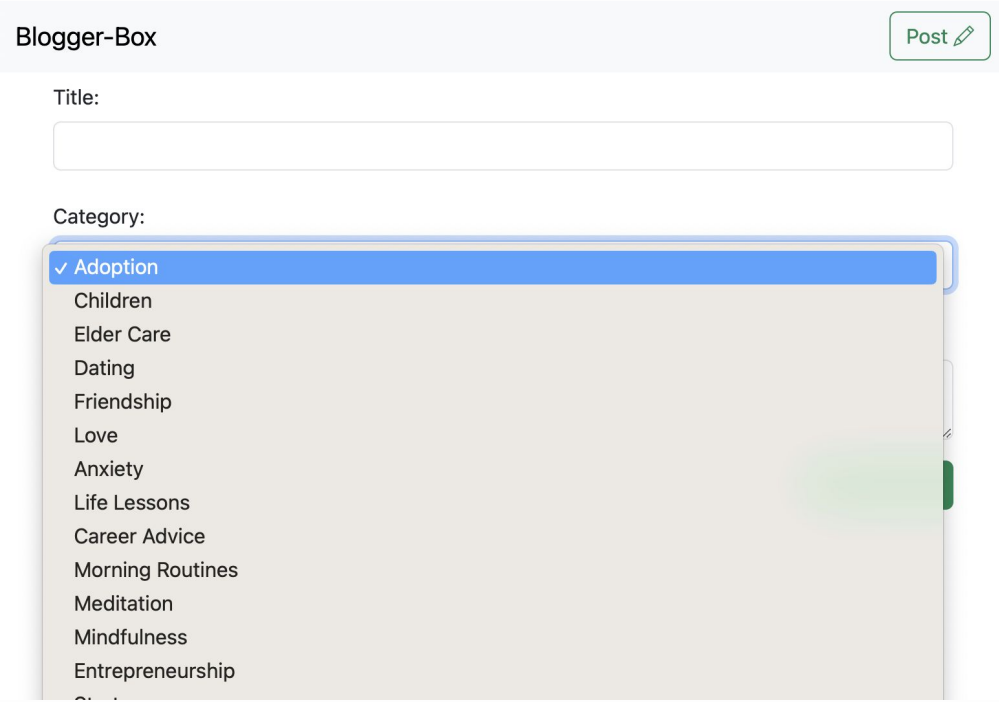
- **Integration of SweetAlert2 Library:**
  - Integrate SweetAlert2 library to provide intuitive feedback messages on creation success/error. Check the Mixin example in the documentation: (https://sweetalert2.github.io/#toast:~:text=You%20can%20easily,%3A)
    - Utilize toast notifications:
      - If the form is invalid upon submission, display a toast with the message **"Please review your post"**.
      - If the form is valid upon submission, display a toast with the message **"Post Submitted Successfully"** and redirect the user to the homepage to view the newly submitted post.

Sync with Github

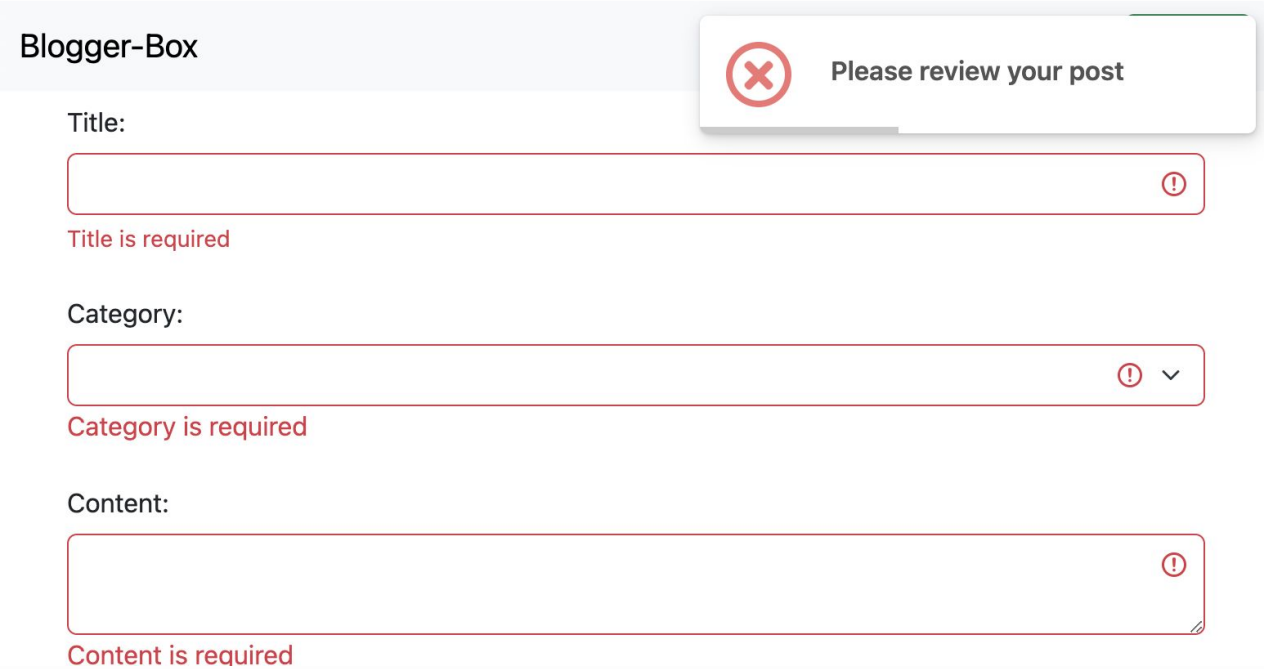
BloggerBox Frontend - Part 2 Expected UI



BloggerBox Frontend - Part 2 Expected UI



BloggerBox Frontend - Part 2 Expected UI



BloggerBox Frontend - Part 2 Expected UI

Blogger-Box

Post

Title:

Supporting Neurodiversity in Children: Celebrating Differences, Embracing Strengths Sup

Title should not exceed 150 characters

Category:

Children

Content:

Cras fringilla, odio sed viverra bibendum, risus enim auctor urna, eget blandit ipsum est vitae enim. Integer vel condimentum libero. Aliquam hendrerit, risus id tincidunt ullamcorper, neque nibh posuere justo, eu lacinia nisi justo in tortor. Donec cursus, eros eget sagittis lacinia, magna magna mattis mauris, at porta purus velit vel tortor. Curabitur fringilla, ligula eleifend mattis bibendum, nulla massa vehicula nisl, non mattis nisi purus sed leo.

Post content should not exceed 2500 characters

Close

Post message

BloggerBox Frontend - Part 2 Expected UI

Blogger-Box

Post

Title:

Supporting Neurodiversity in Children: Celebrating Differences, Embracing Strengths

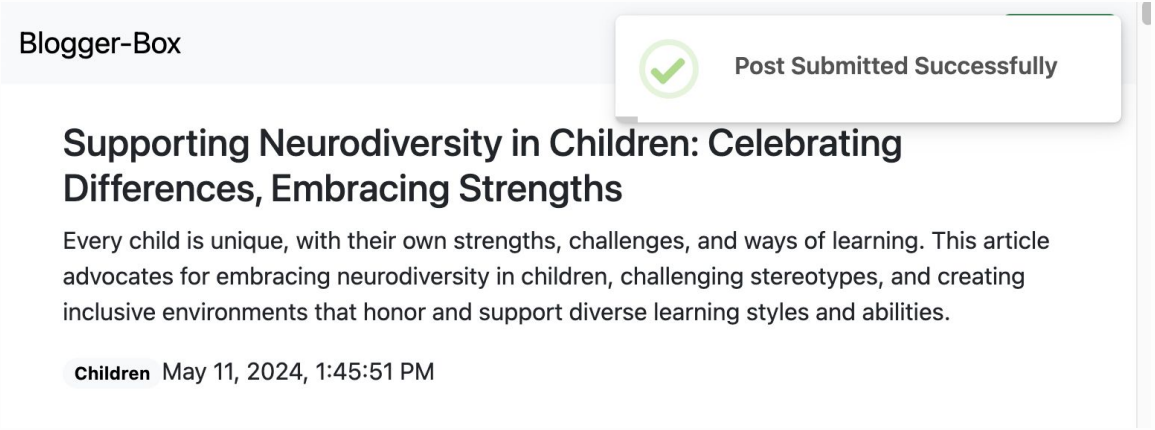
Category:

Children

Content:

Every child is unique, with their own strengths, challenges, and ways of learning. This article advocates for embracing neurodiversity in children, challenging stereotypes, and creating inclusive environments that honor and support diverse learning styles and abilities.

Close

Post message

BloggerBox Frontend - Part 2 Expected UI